DSA Coursework: Final Submission Prep Guide

This document provides a clear, step-by-step checklist to prepare your final three files for submission: `Quick.java`, `output.txt`, and `answers.txt`. [1]

---

1. The Final `Quick.java`

Your `Quick.java` must contain the three optimizations (Shuffle, Median-of-Three, and Insertion Cutoff). Below is the complete, optimized source code based on our experiments. You can copy this into your `Quick.java` file.

Java

```java
import java.util.Random;

public class Quick {

    public Quick(boolean shuffleFirst, boolean useMedianOfThree, int insertionS
        this.shuffleFirst = shuffleFirst;
        this.useMedianOfThree = useMedianOfThree;
        this.insertionSortCutoff = insertionSortCutoff;
    }

    private boolean shuffleFirst;
    private boolean useMedianOfThree;
    private int insertionSortCutoff;

    public void sort(int[] a) {
        if (shuffleFirst) {
            shuffle(a); // Task 2.1: Shuffle before starting
        }
        sort(a, 0, a.length - 1);
        assert Insertion.isSorted(a);
    }

    private void sort(int[] a, int lo, int hi) {
        if (hi <= lo) return;

        // Task 2.3: Switch to insertion sort for small subarrays
        if (hi - lo + 1 <= insertionSortCutoff) {
            Insertion.sort(a, lo, hi);
            return;
        }
```

```
        int j = partition(a, lo, hi);
        sort(a, lo, j-1);
        sort(a, j+1, hi);
        assert Insertion.isSorted(a, lo, hi);
    }

    private int partition(int[] a, int lo, int hi) {
        // Task 2.2: Median-of-three pivot selection
        if (useMedianOfThree) {
            int mid = lo + (hi - lo) / 2;
            int median = medianOfThree(a, lo, mid, hi);
            exchange(a, lo, median);
        }

        int i = lo;
        int j = hi + 1;
        int pivot = a[lo];

        while (a[++i] < pivot)
            if (i == hi) {
                exchange(a, lo, hi);
                return hi;
            }

        while (pivot < a[--j])
            if (j == lo + 1)
                return lo;

        while (i < j) {
            exchange(a, i, j);
            while (a[++i] < pivot);
            while (pivot < a[--j]);
        }

        exchange(a, lo, j);
        return j;
    }

    private static void exchange(int[] a, int i, int j) {
        int swap = a[i];
        a[i] = a[j];
        a[j] = swap;
    }

    private static int medianOfThree(int[] a, int i, int j, int k) {
        boolean x = a[i] < a[j];
        boolean y = a[j] < a[k];
```

```
        boolean z = a[k] < a[i];
        if (x == y) return j;
        if (y == z) return k;
        return i;
    }

    private static void shuffle(int[] a) {
        for (int i = 0; i < a.length; i++) {
            int j = i + random.nextInt(a.length - i);
            exchange(a, i, j);
        }
    }

    private static Random random = new Random(314159265);
}
```

---

2. Generating a Clean `output.txt`

The `output.txt` file should **not** contain all your experimental trials. It should only contain the final report for the algorithms you chose to submit. [1]

How to generate it:

1. **Modify `Bench.java`:** Go to the `main` method and remove all the extra `executionTimeReport` lines you used for testing cutoffs.
2. **Keep only these lines:**

   Java

   ```
   public static void main(final String[] args) {
       // 1. Baseline Quick Sort (to show the "before" state)
       executionTimeReport("Quick: Baseline (F, F, 0)", new Quick(false, false,

       // 2. Your Optimized Quick Sort (using your best cutoff, e.g., 40)
       executionTimeReport("Quick: Optimized (T, T, 40)", new Quick(true, true,

       // 3. The other two algorithms
       executionTimeReport("Insertion.java: insertion sort", Insertion::sort);
       executionTimeReport("Merge.java: merge sort", Merge::sort);
   }
   ```

3. **Run and Save:**
   - Open your terminal/command prompt.

- Run: `javac *.java`
- Run: `java Bench > output.txt`

4. **Verify:** Open `output.txt` and ensure it is a clean file containing only these four reports.

---

3. Completing `answers.txt`

Fill in the template using the data from your clean `output.txt`.

- **Task 1:** Use the per-item growth patterns (Quadratic for Insertion on Random, Linearithmic for Merge, etc.).
- **Task 2.2 (Cutoff):** Explain your systematic method: "I tested cutoffs of 0, 10, 20, 40, and 60. I observed that 40 gave the best performance, and 60 showed a decline in speed, confirming 40 as the optimal crossover point."
- **Task 2.3 (Best Combination):** State that all three optimizations together provided the most robust and fast results. `1`

---

4. Final Packaging

1. Create a folder named after your group (e.g., `omega_group`).
2. Place `Quick.java`, `output.txt`, and `answers.txt` inside it.
3. Zip the folder.
4. Submit the ZIP file.

References

[1] Coursework Repository README